National University of Computer & Emerging Sciences, Karachi

Fall-2026 CY-Department

THEORY PROGRAMMING ASSIGNMENT

Course Code: CS2006	**Course Name: Operating Systems**
Open Date:16th October-2026	**Due Date: 06th November, 2026**

Q1. Solved the exercise of Chapter-4 attached with this assignment in the classroom. Handwritten answers are required. Write codes , make diagrams, do dry run and give reasons , justification where required and necessary,

Q2. You are working for a climate research lab that monitors global temperature anomalies using a network of satellites. The satellites provide daily temperature readings in the form of large NxN matrices for different regions of the Earth.

Your task is to analyze these temperature matrices efficiently using POSIX threads, applying both task parallelism and data parallelism.

Requirements:

- **Multiple matrices from multiple satellites (M matrices)** need to be merged into a **global NxN matrix**.
    - Overlapping regions must be **averaged**.
    - Missing cells (*NaN*) must be interpolated using neighboring values.
- *Multi-level parallelism:*
    - *Satellite-level parallelism:* Each thread handles **one satellite's matrix** and prepares it (interpolation for missing cells).
    - *Region-level parallelism:* Divide the global matrix into **sub-matrices (tiles)**, e.g., *100x100* blocks. Each tile is processed by a separate thread to:
        - Compute local max, min, and average
        - Compute variance and standard deviation
        - Detect anomalies (values deviating >2σ from local mean)
- *Dynamic load balancing:*
    - Some regions have **more data points** due to overlapping satellites. Students must **dynamically assign threads** to regions to avoid idle threads.

- Threads must **safely merge results** into the global statistics without race conditions.

Task Parallelism (Interdependent Tasks)

After data-parallel computation, start these **three interdependent tasks concurrently**:

- **Task A – Hotspot Identification**
  - Detect all cells above *T_hot* and record their positions.
  - Each hotspot must trigger **Task B update** if it's near a coldspot (within 2-cell Manhattan distance).
- **Task B – Coldspot Identification**
  - Detect all cells below *T_cold* and record their positions.
  - Each coldspot must trigger **Task A update** if it's near a hotspot.
- **Task C – Normalization & Weighted Adjustment**
  - Normalize the matrix (0–1).
  - If a cell is both near a hotspot and coldspot (from Tasks A & B), it should get a **weighted adjustment** before normalization.
  - Must run **concurrently** with Tasks A & B but **depend on partial hotspot/coldspot information**.

Create threads that calculate "risk scores" for each cell:

risk_score = normalized_temp * proximity_to_hotspots / (proximity_to_coldspots + 1)

This task must wait for both normalization and hotspot/coldspot identification threads.

Constraints:

- Matrix size: `N = 1000` (large enough to make naive sequential processing inefficient).
- Number of threads for row-wise computation: configurable.
- Thresholds: *T_hot = 35°C, T_cold = -10°C.*

Output:

- Global max, min, mean, and variance.
- Number of hotspots, coldspots, anomalies.

- Sample normalized and weighted-adjusted matrix (5x5 submatrix).

- Top 10 risk cells for extreme weather prediction.

Q3. A **smart hospital emergency department** receives patients of three categories: **Critical , Serious and Normal.** Doctors are also categorized: **Senior Doctors** (treat all patients) and **Junior Doctors** (cannot treat critical patients) . Patients arrive continuously and are handled by **separate patient threads**.

Rules:

1. Critical patients must be treated **immediately if any doctor is free**.
2. If only junior doctors are available, the patient must **wait until a senior doctor arrives**.
3. If 5 serious patients are waiting, one must be **promoted to critical priority**.
4. If a doctor treats **3 normal patients consecutively**, the doctor must **forcefully treat a serious patient next**.
5. Hospital policy states **no patient should wait longer than 30 seconds**.

Task : Your task to prepare the model or design of the problem. Your design contains the number and name of the threads you created, required mutexes ,condition variables , thread roles and deadlock prevention strategy ,complete solutions (logic + POSIX pthread code + proper data structures with proper dry and test cases)

Q4. An airport control system manages **multiple runways** and **incoming/outgoing flights**. Flights can be: Landing (highest priority if fuel low) ,Emergency landing Takeoff and Cargo takeoff . Each **flight request arrives dynamically**.

Constraints:

1. **Emergency flights preempt all others**
2. Only **one flight per runway**
3. Landing flights must be handled before takeoffs if waiting time > threshold
4. Runway maintenance may randomly block a runway
5. Flights waiting too long must **increase priority dynamically**

Implement a **multithreaded scheduler**:

Threads:

- *FlightGeneratorThread*
- *RunwayControllerThreads*

- *EmergencyMonitorThread*

Shared structures:

Priority Queue of Flights
Runway Status Table
Flight Waiting Time Map

Thread Roles:

1. **Flight Generator Thread**
    a. Randomly generates flights (type, arrival time)
    b. Adds flights to **priority queue**
2. **Runway Controller Threads**
    a. Each thread manages a runway
    b. Picks the **highest priority flight** from the queue
    c. Marks runway as busy during flight operation
3. **Emergency Monitor Thread**
    a. Detects emergency flights
    b. Preempts any ongoing landing/takeoff if allowed

Dynamic priority adjustment:

```
if (current_time - flight.arrivalTime > threshold)
  flight.priority++;
```

Emergency preemption:

- If emergency detected, must stop lower-priority flight **safely**

Runway locking:

- *pthread_mutex_lock* to avoid two flights using the same runway

Condition variable for queue:

- Threads wait when no flights are available.

Tasks : Provide complete solutions (logic + POSIX pthread code + proper data structures).

Q5. Write the review of the following research paper titled **: "Parallel processing of E-Atheer algorithm using pthread paradigm" -** Indonesian Journal of Electrical Engineering and Computer Science , Vol. 30, No. 3, June 2023, pp. 1624~1633 , ISSN: 2502-4752, DOI: 10.11591/ijeecs.v30.i3.pp1624-1633 1624 , Journal homepage: http://ijeecs.iaescore.com/

Checking:

Students are required to save a screenshot of each step and define it in word doc with a sequence; the document (***named: k174224_OS_A2.doc***) will be submitted on the GCR. **Students must bring their laptops for the demo of the assignment.** (The date will be given by later)

Note*: All codes must be submitted handwritten.*